arXiv:2502.11620v3 [cs.SE] 1 Jul 2025

Assessing Correctness in LLM-Based Code Generation via Uncertainty Estimation

Arindam Sharma
University of Bristol
Bristol, United Kingdom
arindam.sharma@bristol.ac.uk

Cristina David
University of Bristol
Bristol, United Kingdom
cristina.david@bristol.ac.uk

Abstract

Large Language Models (LLMs) have shown strong performance in code generation, but their outputs lack inherent correctness guarantees. In natural language generation (NLG), uncertainty—often grounded in information theory—has been used as a proxy for output quality. However, state-of-the-art techniques [27, 14, 3] fail to exhibit a correlation between estimated uncertainty and correctness when applied to code. We investigate this discrepancy and identify semantic clustering as the key factor: existing learned embeddings and heuristic metrics such as CodeBLEU fail to capture functional equivalence with sufficient precision to support reliable uncertainty estimation. We propose *symbolic clustering*, a principled alternative based on symbolic execution, enabling precise clustering of semantically equivalent programs. Incorporating symbolic clustering into NLG-derived uncertainty techniques restores their predictive power. We further show that token-level probabilities are not essential—uncertainty estimates assuming uniform distributions perform comparably. Based on this, we introduce a lightweight correctness proxy: symbolic cluster count. Despite its simplicity, it correlates with correctness and supports highly effective abstention policies, achieving a false positive rate below 0.02%.

1 Introduction

LLMs have shown impressive capabilities across a range of software engineering tasks, including code synthesis [29, 43], bug fixing [13, 53], refactoring [9, 42, 40], and translation [52, 33, 12, 39]. However, LLM-generated code lacks inherent correctness guarantees. As a result, correctness is typically assessed by running the code against *external oracles*—test suites, reference solutions, or other expert specifications. While these external techniques can provide evidence of functional correctness, such as passing unit tests, they are not always available. Even when present, they may be insufficiently precise, e.g., unit tests may only cover a limited subset of the code's behaviour.

In contrast, NLG has made notable strides in oracle-free quality estimation by leveraging internal model signals, such as token-level log-probabilities. Among the most effective are information-theoretic methods [14, 27, 3], which estimate a model's *uncertainty* based on the intuition that it peaks when outputs are least informative, using entropy [14, 27] or mutual information [3] as their foundation. Other techniques [28, 10, 35, 38, 8, 51, 18, 23] are either shown to be less effective or rely on costly additional training steps. A key innovation in uncertainty estimation, introduced by Kuhn et al. [27], is *semantic clustering*, which accounts for the diversity of plausible outputs. Rather than estimating uncertainty over individual responses, they group responses into clusters based on semantic equivalence, and compute uncertainty over this coarser space. This shift allows the uncertainty estimates to reflect variation in meaning rather than superficial syntactic differences.

Preprint. Under review.

In this paper, we investigate whether state-of-the-art information-theoretic approaches for estimating model uncertainty—specifically those by Kuhn et al. [27, 14] and Abbasi et al. [3]—generalise to code generation. We evaluate their predictive power by testing for correlation between estimated uncertainty and correctness. Our results show that even when replacing text-specific embeddings with code-specific ones for semantic clustering, uncertainty measures from the original NLG methods fail to correlate with correctness in code generation tasks.

We attribute this to the rigid nature of programs, where small syntactic differences can imply significant behavioural changes or none at all, making learned semantic clustering brittle. To address this, we explore the use of semantic clustering techniques grounded in *symbolic reasoning*. As a first step, we evaluate CodeBLEU [41], a commonly used similarity metric that augments syntactic matching with data flow analysis to approximate semantics [47]. However, CodeBLEU remains a heuristic—it lacks formal reasoning and cannot reliably capture semantic equivalence. In our evaluation, clustering based on CodeBLEU failed to improve the correlation between uncertainty and correctness, confirming that its semantic component is too coarse-grained for this purpose.

This led us to pursue a more precise alternative: *symbolic clustering* via *symbolic execution*, where symbolic execution executes programs over symbolic rather than concrete inputs [6]. By comparing the programs' output expressions, symbolic execution offers us a principled way to identify semantically equivalent programs, even when they vary significantly in syntax. We integrated symbolic clustering into leading NLG-based uncertainty estimation frameworks [3, 14, 27], adapting them to the code generation setting. Unlike their original versions, the adapted methods exhibit a statistically significant correlation between estimated uncertainty and actual correctness. This supports the view that precise semantic reasoning, grounded in formal methods like symbolic execution, is essential for reliable uncertainty estimation in code generation.

Interestingly, we find that token log-probabilities output by the LLM—used by the entropy and mutual information calculations in [27, 14] and [3], respectively—have limited effect: assuming a uniform output distribution performs comparably. This highlights another insight: semantic equivalence, not probabilistic confidence, is the dominant signal for correctness in code generation.

Building on this insight, we propose a much simpler correctness metric: the *symbolic cluster count*. The intuition is straightforward—fewer functionally distinct outputs suggest greater model confidence. Despite its simplicity, this metric shows a correlation with actual correctness, highlighting its effectiveness as a lightweight yet reliable proxy for correctness.

We further leverage our insights into the correlation between uncertainty estimates and correctness to define *abstention policies*, where the model should refrain from generating a response if the response is estimated to be incorrect. Our evaluation shows that, under these policies, the LLM achieves a false positive rate below 0.02%, effectively ensuring that it never outputs incorrect responses, with false negative rates ranging from 11.9% to 20.3%. Notably, this strong performance holds even for the abstention policy based solely on the symbolic cluster count, despite its simplicity.

Contributions.

- We investigate the correlation between LLM uncertainty—computed using state-of-the-art NLG techniques [3, 14, 27]—and correctness in code generation tasks. We find that even when adapted to code (for example by incorporating code-specific learned embeddings or heuristic symbolic metrics like CodeBLEU) these methods fail to exhibit a statistically significant correlation between estimated uncertainty and actual correctness.
- To the best of our knowledge, we are the first to address these limitations by introducing *symbolic clustering*, i.e. a symbolic execution–based approach for clustering semantically equivalent code outputs. This leads to a correlation between uncertainty estimates and functional correctness. Moreover, we show that this correlation holds even when ignoring the LLM's output distribution and assuming an uniform distribution.
- Motivated by our findings, we propose a lightweight alternative correctness metric based solely on *symbolic clusters count*. Despite its simplicity, our experimental evaluation shows it performs on par with more complex, information-theoretic approaches.
- We define *abstention policies*, where the model should refrain from generating a response if the response is estimated to be incorrect. Our evaluation results show that, under these policies, the LLM achieves a false positive rate below 0.02%.
- We conduct an experimental evaluation involving 3 LLMs, the LIVECODEBENCH dataset for code-related tasks [20], 13 correctness proxy techniques, and 5 abstention policies.

2 Background on NLG-Based Uncertainty Estimation Techniques

We selected two recent and well-established state-of-the-art techniques for information-theoretic uncertainty estimation in NLG to adapt and evaluate on code generation tasks. In the first one, Kuhn et al. [14, 27] estimate the uncertainty of a model's prediction as the entropy of the output distribution. In order to account for outputs that differ in form but share the same meaning, they introduce the idea of *semantic entropy*. First, they group sampled model outputs into clusters of semantically equivalent responses, and define $p(c \mid x) = \sum_{s \in c} p(s \mid x)$ as the total probability mass of cluster c under the model's output distribution. They then measure uncertainty over meanings (rather than raw token sequences) by taking the entropy of this cluster-level distribution: $\mathrm{SE}(x) = -\sum_c p(c \mid x) \log p(c \mid x)$. This notion of semantic entropy offers a more accurate measure of uncertainty in free-form NLG, as it collapses paraphrases into a single representation before computing entropy.

In the second work, Abbasi et al. [3] further distinguish between "epistemic" uncertainty (caused by incomplete or incorrect knowledge) and "aleatoric" uncertainty (resulting from inherent randomness, such as multiple valid answers). Intuitively, only epistemic uncertainty serves as a meaningful indicator of the correctness of the model's response. To isolate epistemic uncertainty, they propose an iterative prompting technique in which the model is repeatedly queried, with its prior responses incorporated into the context. This method assesses the mutual information between successive responses to capture their dependencies: a high degree of dependency indicates elevated epistemic uncertainty, revealing the model's uncertainty about the ground truth. Similar to the work of Kuhn et al. , Abbasi et al. cluster responses into semantic equivalence classes, ensuring that the mutual information reflects semantic relationships.

More precisely, in order to isolate epistemic uncertainty, Abbasi et al. [3] quantify it as the mutual information of the pseudo-joint distribution over n successive responses:

$$I(\tilde{\mu}) = \sum_{y_{1:n}} \tilde{\mu}(y_{1:n} \mid x) \log \frac{\tilde{\mu}(y_{1:n} \mid x)}{\prod_{i=1}^{n} \tilde{\mu}_i(y_i \mid x)} \tag{4.5}$$

Here, $\tilde{\mu}(y_{1:n} \mid x) = \prod_{i=1}^{n} \mu\big(Y_i \mid F_{i-1}(x, Y_{1:i-1})\big)$ is the pseudo-joint distribution obtained by iteratively prompting the model (with $F_{i-1}(x, Y_{1:i-1})$ denoting the context composed of the original query x plus all previous responses $Y_1, \ldots, Y_{i-1}$), $\tilde{\mu}_i(y_i \mid x) = \sum_{y_{\neg i}} \tilde{\mu}(y_{1:n} \mid x)$ is its marginal for the i-th response, and $I(\tilde{\mu})$ measures how much knowing one response reduces uncertainty about the others.

3 Motivating Example

```python
def solve(nums):
    max_num = max(nums)
    expected_length = max_num + 1
    if len(nums) != expected_length:
        return False
    count = {}
    for num in nums:
        if num in count:
            count[num] += 1
        else:
            count[num] = 1
    for num in range(1, max_num):
        if count.get(num, 0) != 1:
            return False
    if count.get(max_num, 0) != 2:
        return False
    return True
```

(a) Snippet 1

```python
def solve(nums):
    n = max(nums)
    base = list(range(1, n)) + [n, n]
    if len(nums) != len(base):
        return False
    count_nums = {}
    count_base = {}
    for num in nums:
        count_nums[num] = count_nums.get(num, 0) + 1
    for num in base:
        count_base[num] = count_base.get(num, 0) + 1
    return count_nums == count_base
```

(b) Snippet 2

```python
def solve(nums):
    max_num = max(nums)
    n = max_num
    expected_length = n + 1
    if len(nums) != expected_length:
        return False
    count = {}
    for num in nums:
        if num != n and nums.count(num) != 1:
            return False
        if num == n and nums.count(num) != 2:
            return False
    for i in range(1, n):
        if i not in nums:
            return False
    return True
```

(c) Snippet 3

Figure 1: Three code snippets from the `DeepSeek-R1` [15], all correct, semantically equivalent.

We use problem 2892 from LIVECODEBENCH, where the task is to determine whether the given array is a "good" array or not. An array is considered good if it is a permutation of an array base[n] where base[n] = [1, 2, ..., n - 1, n, n] (in other words, it is an array of length n + 1 which contains 1 to n - 1 exactly once, plus two occurrences of n). For example, base[1] = [1, 1] and base[3] = [1, 2, 3, 3].

We sample three programs generated by `DeepSeek-R1`. Although they differ syntactically in their loop structures and placement of `if` statement guards, they are semantically equivalent and correctly solve the task.

Clustering with learned embeddings. Despite this, semantic clustering using DEBERTA embeddings [16] (as used by Kuhn et al.[27]) and OPENAI's GPT `text-embedding-ada-002` embeddings (shown to outperform prior code embeddings like `code-search-babbage-001` and `code-search-ada-001` [37]) fails to recognise their equivalence, placing each snippet in a separate cluster.

Clustering with CODEBLEU. When applying CODEBLEU-based clustering, the similarity scores between snippets are low: CODEBLEU(Snippet 1a, Snippet 1b) = 0.49, CODEBLEU(Snippet 1a, Snippet 1c) = 0.47, and CODEBLEU(Snippet 1b, Snippet 1c) = 0.28. For context, heuristic thresholds in the range of 0.7-0.8 are often used to indicate high similarity. As such, CODEBLEU fails to cluster these semantically equivalent programs, placing each in its own distinct cluster.

Symbolic clustering. This highlights the need for a more principled approach. Symbolic execution [24, 6] offers such a technique: it analyses programs by treating inputs as symbolic variables rather than concrete values. As the program executes, it incrementally constructs symbolic expressions representing arithmetic computations and accumulates path constraints at conditional branches. The result is a precise, algebraic characterisation of the program's behaviour that uses symbolic expressions and logical formulas rather than actual values or executions. By comparing the behaviour of the three snippets as captured by the symbolic input-output relations, we conclude that there are all semantically equivalent, meaning that they go into the same cluster. Due to the symbolic value for the length of the input array $nums$, which means that we don't know how many times we need to unwind loops, we perform a bounded equivalence check as explained in Section 4.

Impact of clustering on uncertainty estimation. With symbolic clustering, which correctly groups all three snippets into the same cluster, the uncertainty scores reported by Kuhn et al. [27] and Abbasi et al. [3] are zero, as expected for semantically equivalent and correct outputs. In contrast, all other clustering methods yield non-zero uncertainty. For instance, Kuhn et al. 's method produces an uncertainty score of 1.49 when using the GPT `text-embedding-ada-002` embeddings (which are also tailored for code [37]). As a reference point, the abstention threshold for this method in our experimental evaluation is 0.82 (see Section 5.1 for how this threshold is derived), meaning that the model would incorrectly abstain from outputting any of these correct solutions.

We also experiment with replacing the model-derived probability distribution (e.g., 0.48, 0.29, 0.23 for `DeepSeek-R1`) with a uniform distribution. Even under this simplified setting, uncertainty estimates based on symbolic clustering remain zero, underscoring the symbolic clustering's robustness to changes in the underlying probability distribution.

Symbolic cluster count. Using the symbolic cluster count, which is 1 in this case, as all snippets fall into the same cluster, the abstention policy permits outputting the result, since the count is below the corresponding abstention threshold.

4 Estimating Uncertainty with Symbolic Clustering

In the domain of code generation, program equivalence has a precise definition: two programs are considered equivalent if they produce identical behaviour for all possible inputs. Consequently, a domain-specific equivalence check is required. In this paper, we base the semantic equivalence check on *symbolic execution*, where, instead of executing a program with concrete inputs, *symbolic variables* are used to represent inputs, generating constraints that describe the program's behaviour across all possible input values [6].

The particular flavor of symbolic execution we use in this work is inspired by the lightweight *peer architecture* described in Bruni et al. [5]. Unlike traditional approaches that require building

a standalone symbolic interpreter, this architecture embeds the symbolic execution engine as a lightweight library operating alongside the target program. Their design is based on the insight that languages that provide the ability to dynamically dispatch primitive operations (e.g., Python) allow symbolic values to behave as native values and be tracked at runtime.

Symbolic execution typically traverses the program's control flow graph, maintaining a symbolic state consisting of *path constraints* (i.e. logical conditions that must be satisfied for a given execution path to be feasible) and *symbolic expressions* (i.e. representations of program variables as functions of the symbolic inputs).

Algorithm 1 Symbolic Clustering

Require: Set of generated code snippets $\{s^{(1)}, \ldots, s^{(M)}\}$
Ensure: Clusters of semantically equivalent snippets $C = \{c_1, c_2, \ldots, c_k\}$
1: Initialise an empty cluster set $C \leftarrow \emptyset$, and an equivalence map $E \leftarrow \emptyset$
2: **for** each snippet $s^{(i)}$ **do**
3: **if** $s^{(i)}$ is invalid **then**
4: $E[s^{(i)}] \leftarrow \{ s^{(i)} \}$ ▷ Assign invalid snippet to its own equivalence class
5: **end if**
6: **end for**
7: **for** each pair of valid snippets $(s^{(i)}, s^{(j)})$ **do**
8: Perform symbolic execution on $s^{(i)}$ and $s^{(j)}$ to extract traces $T(s^{(i)})$ and $T(s^{(j)})$
9: **if** $T(s^{(i)}) \equiv T(s^{(j)})$ **then**
10: $E[s^{(i)}] \leftarrow E[s^{(i)}] \cup \{ s^{(j)} \}$
11: $E[s^{(j)}] \leftarrow E[s^{(j)}] \cup \{ s^{(i)} \}$
12: **end if**
13: **if** $s^{(i)} \sim s^{(j)}$ and $s^{(j)} \sim s^{(k)}$ for some $s^{(k)}$ **then** ▷ Enforce transitivity of equivalences
14: $E[s^{(i)}] \leftarrow E[s^{(i)}] \cup \{ s^{(k)} \}$
15: $E[s^{(k)}] \leftarrow E[s^{(k)}] \cup \{ s^{(i)} \}$
16: **end if**
17: **end for**
18: Identify equivalence classes in E to form final clusters C
19: **return** C

Symbolic clustering. Algorithm 1 illustrates how to cluster code snippets based on their functional semantics, with an additional check for invalid snippets. We first create empty structures for storing the final clusters (C) and an equivalence map (E) to track relationships (line 1). Next, in the *invalid snippet handling phase* (line 2), each code snippet $s^{(i)}$ is examined and, if it is detected to be syntactically invalid, it is immediately placed in its own equivalence class in E and thus isolated from further consideration. In the *pairwise comparison phase* (line 7), each pair of *valid* snippets $(s^{(i)}, s^{(j)})$ is symbolically executed. For each snippet $s^{(i)}$, symbolic execution produces a *set of traces* $T\big(s^{(i)}\big) = \big\{\tau_1, \tau_2, \ldots\big\}$, where each trace $\tau \in T(s^{(i)})$ comprises of a path constraint, and a symbolic state, which encodes the expressions for all variables encountered along that path. We declare two snippets $s^{(i)}$ and $s^{(j)}$ *semantically equivalent* exactly when their trace sets are the same (line 9): $T\big(s^{(i)}\big) \equiv T\big(s^{(j)}\big)$, i.e. for *every* trace τ in $T(s^{(i)})$, there is a matching trace in $T(s^{(j)})$ and vice versa. Two traces τ_1 and τ_2 match if there is no counterexample input that leads to distinguishing output states for τ_1 and τ_2. Since program equivalence is undecidable in general, we perform a bounded equivalence check, where we verify that no counterexample input exists when exploring traces up to a given depth.

If the snippets are semantically equivalent, we add each snippet to the other's equivalence class (line 11). To maintain consistency, we also enforce transitivity: whenever $s^{(i)} \sim s^{(j)}$ and $s^{(j)} \sim s^{(k)}$, we merge $s^{(i)}$ and $s^{(k)}$. Finally, the equivalence map E is processed to extract the clusters themselves (line 18), and the resulting set of clusters is returned (line 19).

Uncertainty estimation for code generation. The original NLG techniques by Kuhn et al. [27] and Abbasi et al. [3] involve sampling responses from an LLM, extracting token-level log-probabilities, and applying softmax-style normalization to interpret these as a probability distribution. However, in contrast to the original evaluations (where the outputs were typically short, one-word answers from question-answering datasets) code generation yields much longer responses. Since the probability

of a response is computed as the joint probability of its tokens, it decays exponentially with length, often leading to *numerical underflow*. In such cases, the softmax normalisation can produce NaNs, rendering uncertainty estimates unusable.

To address this, we explore two strategies for approximating the output distribution of LLM responses. First, following prior work [36, 26, 34], we normalise the log-probability of each program by its length, mitigating the bias against longer outputs and preventing underflow. Secondly, as a more radical simplification, we ignore log-probabilities altogether and assume a uniform distribution over the n sampled responses, assigning each a probability of $1/n$. Under this uniform-distribution simplification, we set $p(s \mid x) = \frac{1}{n}$ and $p(c \mid x) = \sum_{s \in c} p(s \mid x) = \frac{|c|}{n}$. Hence the semantic entropy becomes $\mathrm{SE}_{\mathrm{uniform}}(x) = -\sum_c \frac{|c|}{n} \log \frac{|c|}{n}$.

Likewise, the mutual information over n iterative responses simplifies to

$$I_{\mathrm{uniform}}(x) = \sum_{c_{1:n}} p(c_{1:n} \mid x) \log \frac{\tilde{p}(c_{1:n} \mid x)}{\prod_{i=1}^{n} p_i(c_i \mid x)},$$

where $p(c_{1:n} \mid x) = \prod_{i=1}^{n} \tilde{p}(c_i \mid F_{i-1}(x,\, c_{1:i-1})) = \prod_{i=1}^{n} \frac{|c_i|}{n}$ and $p_i(c_i \mid x) = \sum_{c_{\neg i}} \tilde{p}(c_{1:n} \mid x)$.

Interestingly, the uniform approximation performs comparably to the use of actual log-probabilities, reinforcing our central finding: semantic equivalence, not probabilistic confidence, is the dominant signal for estimating uncertainty in code generation.

Motivated by this, we propose a much simpler metric: the *symbolic cluster count*. Intuitively, a smaller number of distinct semantic outputs reflects higher model confidence. Despite its simplicity, this metric performs on par with more complex, information-theoretic approaches—provided the clustering is powered by symbolic execution.

The adaptations of the techniques by Kuhn et al. and Abbasi et al. to the code generation domain are detailed in Appendix Sections A and B, respectively.

5 Evaluation

LLM selection. We selected the open-source model `DeepSeek-R1` [15] as it represents the latest state-of-the-art among publicly available models. To complement this, we included `gpt-3.5-turbo-instruct` [2] from OpenAI [1], the most recent OpenAI model that exposes token-level `logprobs` via its API, making it well-suited for evaluating information-theoretic uncertainty techniques. Finally, to evaluate methods that do not rely on log-probabilities, such as those assuming a uniform distribution or based on symbolic cluster counts, we also used `claude-3-7-sonnet` [4], a state-of-the-art proprietary model that does not expose log-probabilities.

Dataset selection. We evaluate our techniques using LiveCodeBench [20], a contamination-free benchmark for code-related tasks comprising 831 problems, categorised as *Easy* (239), *Medium* (332), and *Hard* (260). Sourced from LeetCode, AtCoder, and CodeForces, each problem includes a natural language description and I/O test cases. We use the natural language description in the query provided to the LLM, and the test cases for evaluating our techniques, as detailed later in the section. Table 1 presents statistics describing the behaviour of the models evaluated in this study (`gpt-3.5-turbo-instruct`, `DeepSeek-R1`, and `claude-3-7-sonnet`) on LiveCodeBench.

Table 1: Solution statistics for `gpt-3.5-turbo-instruct`, `DeepSeek-R1` and `claude-3-7-sonnet` across various problem difficulty levels from LiveCodeBench.

Metric	`gpt-3.5-turbo-instruct`			`DeepSeek-R1`			`claude-3-7-sonnet`		
	Easy	**Medium**	**Hard**	**Easy**	**Medium**	**Hard**	**Easy**	**Medium**	**Hard**
Pass Rate	50%	30%	23%	84%	74%	52%	70%	61%	43%
Average Lines of Code	13	20	22	12	29	36	12	31	38
Average Tokens	198	207	218	81	188	212	74	192	231

Experimental setup. Our experiments were conducted on a machine running `Ubuntu 20.04.5 LTS (Focal Fossa)` with one `NVIDIA A100 GPU (80GB)`. For the approach based on Kuhn et

al. [27], we ask the LLM to generate 5 responses for each problem along with their respective `log-probabilities`. On top of that, for the approach based on Abbasi et al. [3], we perform 2 iterations of prompting for each of the 5 generated responses. The first iteration involves querying the model with the original prompt, while the second iteration uses a concatenated prompt, combining the original prompt and the response from the first iteration. In order to perform symbolic execution for clustering, we use `Crosshair` [7] with a per condition timeout of 5 s and the same per path timeout of 5 s. In addition to this, we also impose an overall timeout of 10 s for each pair of programs that is being checked for equivalence. If a counterexample showing the difference in behaviour is not found within this timeout, we assume that the programs belong in the same cluster.

We evaluate three main categories of techniques: SE—based on the semantic entropy method proposed by Kuhn et al. [27, 14]; MI—based on the mutual information approach by Abbasi et al. [3]; and CC—the cluster count metric. For SE and MI, we explore several variants depending on how semantic clustering is performed and how response probabilities are treated. We use the following suffixes to distinguish them:

- NLG: the original implementation using natural language embeddings;
- CODEEMBED: using GPT `text-embedding-ada-002` embeddings for clustering, which outperform prior code models like `code-search-babbage-001` and `code-search-ada-001` [37];
- CODEBLEU: using CODEBLEU for clustering;
- SYMB: using symbolic clustering;
- SYMBUNIFORM: using symbolic clustering while assuming a uniform distribution over outputs.

For CC, we evaluate CODEEMBED and SYMB, where CC-SYMB is our proposed symbolic cluster count metric.

For each problem in the benchmark, we compute:

- The corresponding *uncertainty score* for all the variants of SE and MI.
- The *number of clusters* obtained for all variants of CC.
- The *probablity of the top-ranked response* for LLM-PROBABILITY.
- The *correctness score*, which is calculated as follows: the top-ranked response generated by the LLM (as determined by the API's ranking) is executed against the benchmark's test cases. The percentage of test cases successfully passed is recorded as the correctness score. A generous timeout of 5 s is applied to each test case. If the candidate solution exceeds this timeout, the test case is considered "failed".

Results. In Table 2, we compute the Pearson correlation between the result computed by each of the techniques in the first column and the correctness scores (for a particular model and dataset). We report the Pearson correlation coefficient and p-value, with statistically significant results highlighted. For `claude-3-7-sonnet`, which does not expose log-probabilities, the evaluation is limited to SE-SYMBUNIFORM and CC-SYMB, and shown in Table 4 in Appendix.

5.1 Discussion of Results

Symbolic clustering enables NLG-based uncertainty estimation methods to perform effectively in code generation. As shown in Table 2, the SE and MI techniques fail to exhibit statistically significant correlation between estimated uncertainty and correctness when applied to code generation—even when adapted with OpenAI's GPT `text-embedding-ada-002` embeddings, which are tailored for code [37] or heuristic similarity metrics such as CODEBLEU. Notably, both techniques only achieve statistically significant correlation when paired with symbolic clustering (SE-SYMBNORM, SE-SYMBUNIFORM, MI-SYMBNORM, and MI-SYMBUNIFORM), highlighting the importance of precise semantic equivalence analysis in this domain.

Semantic equivalence among the LLM responses is a strong indicator of correctness. When relying on log-probabilities computed by the LLM (row corresponding to LLM-PROBABILITY in Table 2), we find no statistically significant correlation with correctness, indicating that token-level probabilities are insufficient proxies for correctness in code generation. In contrast, symbolic clustering provides a much stronger signal. Even when assuming a uniform distribution over LLM responses, SE-SYMBUNIFORM and MI-SYMBUNIFORM still yield a statistically significant correlation with correctness.

Lightweight metrics based on symbolic clustering can still yield effective correctness estimates. Pushing this abstraction further, CC-SYMB, which relies solely on the number of symbolic clusters,

Table 2: Correlation Results — Pearson coefficient (p-value) for `gpt-3.5-turbo-instruct` and `DeepSeek-R1`. Statistically significant results are highlighted in **bold**.

Technique	`gpt-3.5-turbo-instruct`			`DeepSeek-R1`		
	Easy	**Medium**	**Hard**	**Easy**	**Medium**	**Hard**
SE-NLG	−0.04(0.5792)	−0.02(0.6520)	−0.01(0.2941)	−0.06(0.2762)	−0.06(0.1651)	−0.08(0.9289)
SE-CODEEMBED	−0.10(0.1083)	0.07(0.1867)	−0.02(0.1945)	−0.09(0.1631)	−0.08(0.1409)	−0.08(0.1567)
SE-CODEBLEU	−0.21(0.0892)	−0.18(0.0945)	−0.17(0.0888)	−0.13(0.0512)	−0.11(0.0904)	−0.10(0.1256)
SE-SYMBNORM	**−0.56(0.0001)**	**−0.52(0.0002)**	**−0.49(0.0002)**	**−0.45(0.0002)**	**−0.40(0.0001)**	**−0.41(0.0008)**
SE-SYMBUNIFORM	**−0.51(0.0001)**	**−0.48(0.0001)**	**−0.47(0.0002)**	**−0.45(0.0002)**	**−0.41(0.0001)**	**−0.40(0.0007)**
MI-NLG	−0.02(0.6353)	−0.04(0.1828)	−0.07(0.4748)	−0.05(0.7674)	−0.04(0.8372)	−0.05(0.5365)
MI-CODEEMBED	−0.024(0.9123)	−0.056(0.7735)	−0.081(0.6744)	−0.09(0.3245)	−0.12(0.7823)	−0.02(0.5456)
MI-CODEBLEU	−0.12(0.7865)	−0.11(0.7698)	−0.08(0.8213)	−0.21(0.5443)	−0.17(0.5242)	−0.12(0.8728)
MI-SYMBNORM	**−0.22(0.0292)**	**−0.24(0.0132)**	**−0.19(0.0145)**	**−0.37(0.0261)**	**−0.38(0.0121)**	**−0.33(0.0187)**
MI-SYMBUNIFORM	**−0.21(0.0002)**	**−0.18(0.0004)**	**−0.19(0.0008)**	**−0.33(0.0001)**	**−0.34(0.0004)**	**−0.32(0.0003)**
CC-CODEEMBED	−0.01(0.8385)	0.05(0.3076)	−0.08(0.2787)	−0.11(0.1060)	−0.02(0.6685)	−0.01(0.7844)
CC-SYMB	**−0.51(0.0002)**	**−0.44(0.0003)**	**−0.49(0.0003)**	**−0.46(0.0001)**	**−0.41(0.0002)**	**−0.40(0.0004)**
LLM-PROBABILITY	0.17(0.0200)	0.11(0.8236)	0.09(0.3838)	0.18(0.8221)	0.11(0.2762)	0.13(0.6353)

also correlates with correctness. Crucially, this correlation hinges on the precision of clustering: CC-CODEEMBED doesn't exhibit statistically significant results. This result is particularly relevant because many modern LLMs do not expose reliable log-probabilities. For instance, it allows us to evaluate CC-SYMB and SE-SYMBUNIFORM—methods that disregard the LLM log-probabilities—for `claude-3-7-sonnet` (results in Table 4, Appendix), showing that both achieve a statistically significant correlation with correctness.

Our results hold across different models and different problem complexities.

Abstention policies. We envision uncertainty scores being utilised as part of an abstention policy, similar to the approach described by Abbasi et al. [3]. In this policy, an uncertainty threshold is set such that LLM responses with scores below the threshold are assumed to be functionally corrent, while those above the threshold are considered incorrect. For the latter, the LLM may abstain from presenting these responses to the user. LLM responses are classified as correct if their correctness score exceeds a predefined threshold and incorrect otherwise. We used a correctness score threshold of 90%. To determine the corresponding *uncertainty threshold* for the abstention policy, we select the value that maximises correctness accuracy. We split the dataset 50/50 into training and validation. To mitigate overfitting and reduce bias, we employed 2-fold cross-validation, alternating between training and validation for each fold.

For our experimental evaluation, we focus on the best-performing uncertainty estimation techniques—SE-SYMBNORM, SE-SYMBUNIFORM, and CC-SYMB —while using SE-CODEEMBED and LLM-PROBABILITY as baselines for comparison. Table 3 shows that SE-SYMBNORM, SE-SYMBUNIFORM, and CC-SYMB, all using symbolic clustering, achieve high accuracy and extremely low false positive rates, i.e. the fraction of incorrect responses mistakenly shown to the user. This conservative acceptance of LLM outputs is crucial for code generation, where incorrect code poses safety risks. False negative rates (i.e., correct responses unnecessarily filtered out) range from 11.9% to 20.3%, an acceptable trade-off for enhanced reliability.

SE-CODEEMBED and LLM-PROBABILITY performed significantly worse across accuracy, false positive, and false negative rates, rendering them ineffective in practice. In particular, SE-CODEEMBED 's poor results highlight that simply replacing NLG-specific embeddings with code-specific ones is insufficient for adapting uncertainty estimation techniques to code generation and building effective abstention policies.

Limitations. A key limitation of our approach lies in the bounded nature of symbolic execution, particularly with respect to loops. To ensure tractability, we unroll loops and explore program paths only up to the timeout described in the experimental setup. While this enables efficient analysis, it introduces the risk of false positives in equivalence checking: two code snippets may be deemed

Table 3: Abstention metrics

Technique	gpt-3.5-turbo-instruct			DeepSeek-R1		
	Accuracy	**False Pos.**	**False Neg.**	**Accuracy**	**False Pos.**	**False Neg.**
SE-SYMBNORM	81.2 %	0.01 %	17.5 %	84.6 %	0.01 %	12.4 %
SE-SYMBUNIFORM	77.8 %	0.01 %	20.3 %	81.3 %	0.01 %	17.6 %
CC-SYMB	79.1 %	0.02 %	16.6 %	83.1 %	0.01 %	11.9 %
SE-CODEEMBED	58.2 %	10.1 %	24.5 %	59.6 %	11.2 %	23.4 %
LLM-PROBABILITY	36.5 %	26.8 %	37.3 %	38.5 %	21.4 %	32.1 %

semantically equivalent simply because their behavioural differences only manifest beyond the unrolling limit. Another important limitation is that we only focus on the NLG-based uncertainty estimation techniques proposed by Kuhn et al. [27, 14] and Abbasi et al. [3]. We mitigated this by selecting two recent state-of-the-art methods that have received significant attention and citations in the literature.

6 Related Work

Numerous approaches have been proposed for LLM evaluation in NLG. One class of methods involves training dedicated evaluators [49, 48] or using proprietary LLMs such as ChatGPT [45, 25]. However, these approaches are static, opaque, or biased [46], and are outside the scope of our oracle-free setting. Within NLG, oracle-free evaluation has leveraged both self-assessment [22, 50, 30, 23] and internal signals—such as token-level log-probabilities—to estimate uncertainty [3, 14, 27, 34].

In this paper, we focus on the most recent state-of-the-art approaches based on entropy [14, 27] and mutual information [3], which have demonstrated strong performance in NLG. Other techniques [28, 10, 35, 38, 8, 51, 18, 23] are either shown to be less effective or rely on costly additional training steps. In another related work, Jiang et al. [21] show that naive log-likelihoods are weak correctness proxies.

While all the aforementioned works focus on natural language, most code generation studies rely on external oracles to assess the quality of LLM-generated code. For example, works such as [32] and [44] conduct large-scale evaluations of LLM-generated code against existing correctness oracles. Similarly, Honarvar et al. [17] introduce a test suite expansion technique to generate additional code generation problems with oracles, enabling a more comprehensive evaluation of LLM performance.

Another closely related area is hallucination detection, where studies such as [31] and [11] focus on evaluating LLMs using problems specifically designed to expose inconsistencies and hallucinations. Liu et al.[31] establish a taxonomy of hallucinations in LLM-generated code and propose a benchmark to evaluate the ability of code LLMs to recognise hallucinations. Eghbali and Pradel[11] introduce a technique that grounds LLM predictions by retrieving relevant API references and iteratively refining prompts with increasingly relevant context, leading to more reliable LLM-generated code.

Closest to our work, Huang et al. [19] explore lightweight uncertainty proxies (e.g., CODEBLEU) for code generation, but find them ineffective. Unlike prior work, we introduce symbolic clustering and propose metrics that show correlation with correctness—making us, to our knowledge, the first to identify effective oracle-free uncertainty estimators in code generation.

7 Conclusion

Our investigation reveals that existing uncertainty estimation techniques from natural language generation fail to generalise to code generation unless paired with precise semantic clustering. By introducing symbolic execution–based clustering, we restore the correlation between estimated uncertainty and correctness. This enables both more accurate uncertainty estimation and the definition of abstention policies with near-zero false positives. We further demonstrate that a simple symbolic cluster count offers a lightweight yet effective proxy for correctness, providing practical value even in settings where token-level probabilities are unavailable.

References

[1] OpenAI. https://openai.com. Accessed: January 19, 2025.

[2] gpt-3.5-turbo-instruct. https://platform.openai.com/docs/models/gpt-3-5, 2023. Accessed: January 19, 2025.

[3] Yasin Abbasi-Yadkori, Ilja Kuzborskij, András György, and Csaba Szepesvari. To believe or not to believe your llm: Iterative prompting for estimating epistemic uncertainty. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024.

[4] Anthropic. Claude 3: A large language model. https://www.anthropic.com/claude, 2023.

[5] Alessandro Maria Bruni, Tim Disney, and Cormac Flanagan. A peer architecture for lightweight symbolic execution. 2011. URL https://api.semanticscholar.org/CorpusID:61565931.

[6] Cristian Cadar, Daniel Dunbar, and Dawson Engler. Klee: unassisted and automatic generation of high-coverage tests for complex systems programs. In *Proceedings of the 8th USENIX Conference on Operating Systems Design and Implementation*, OSDI'08, page 209–224, USA, 2008. USENIX Association.

[7] Phillip S. Chaney. Crosshair: A python analysis tool for exploring the correctness of programs. https://github.com/pschanely/CrossHair, 2023. Accessed: January 19, 2025.

[8] Jeremy R Cole, Michael JQ Zhang, Daniel Gillick, Julian Martin Eisenschlos, Bhuwan Dhingra, and Jacob Eisenstein. Selectively answering ambiguous questions. *arXiv preprint arXiv:2305.14613*, 2023.

[9] Cristina David, Pascal Kesseli, Daniel Kroening, and Hanliang Zhang. Quantifying the benefits of code hints for refactoring deprecated java apis. In *Companion Proceedings of the 33rd ACM SIGSOFT International Symposium on the Foundations of Software Engineering (FSE 2025 - Industry Track)*, Trondheim, Norway, 2025. ACM. Industry Track.

[10] Shrey Desai and Greg Durrett. Calibration of pre-trained transformers. *arXiv preprint arXiv:2003.07892*, 2020.

[11] Aryaz Eghbali and Michael Pradel. De-hallucinator: Iterative grounding for llm-based code completion. *arXiv preprint arXiv:2401.01701*, 2024.

[12] Hasan Ferit Eniser, Hanliang Zhang, Cristina David, Meng Wang, Maria Christakis, Brandon Paulsen, Joey Dodds, and Daniel Kroening. Towards translating real-world code with llms: A study of translating to rust. *arXiv preprint arXiv:2405.11514*, 2024.

[13] Zhiyu Fan, Xiang Gao, Martin Mirchev, Abhik Roychoudhury, and Shin Hwei Tan. Automated repair of programs from large language models. In *Proceedings of the 45th International Conference on Software Engineering*, ICSE '23, page 1469–1481. IEEE Press, 2023. ISBN 9781665457019. doi: 10.1109/ICSE48619.2023.00128. URL https://doi.org/10.1109/ICSE48619.2023.00128.

[14] Sebastian Farquhar, Jannik Kossen, Lorenz Kuhn, and Yarin Gal. Detecting hallucinations in large language models using semantic entropy. *Nature*, 630(8017):625–630, 2024.

[15] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

[16] Pengcheng He, Xiaodong Liu, Jianfeng Gao, and Weizhu Chen. Deberta: Decoding-enhanced bert with disentangled attention. *arXiv preprint arXiv:2006.03654*, 2020.

[17] Shahin Honarvar, Mark van der Wilk, and Alastair Donaldson. Turbulence: Systematically and automatically testing instruction-tuned large language models for code, 2024. URL https://arxiv.org/abs/2312.14856.

[18] Bairu Hou, Yujian Liu, Kaizhi Qian, Jacob Andreas, Shiyu Chang, and Yang Zhang. Decomposing uncertainty for large language models through input clarification ensembling. *arXiv preprint arXiv:2311.08718*, 2023.

[19] Yuheng Huang, Jiayang Song, Zhijie Wang, Shengming Zhao, Huaming Chen, Felix Juefei-Xu, and Lei Ma. Look before you leap: An exploratory study of uncertainty analysis for large language models. *IEEE Transactions on Software Engineering*, 2025.

[20] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code, 2024. URL https://arxiv.org/abs/2403.07974.

[21] Zhengbao Jiang, Jun Araki, Haibo Ding, and Graham Neubig. How can we know when language models know? on the calibration of language models for question answering. *Transactions of the Association for Computational Linguistics*, 9:962–977, 2021.

[22] Daniel D. Johnson, Daniel Tarlow, David Duvenaud, and Chris J. Maddison. Experts don't cheat: Learning what you don't know by predicting pairs. In *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024*. OpenReview.net, 2024. URL https://openreview.net/forum?id=AVEc9LvSlO.

[23] Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, Dawn Drain, Ethan Perez, Nicholas Schiefer, Zac Hatfield-Dodds, Nova DasSarma, Eli Tran-Johnson, Scott Johnston, Sheer El Showk, Andy Jones, Nelson Elhage, Tristan Hume, Anna Chen, Yuntao Bai, Sam Bowman, Stanislav Fort, Deep Ganguli, Danny Hernandez, Josh Jacobson, Jackson Kernion, Shauna Kravec, Liane Lovitt, Kamal Ndousse, Catherine Olsson, Sam Ringer, Dario Amodei, Tom Brown, Jack Clark, Nicholas Joseph, Ben Mann, Sam McCandlish, Chris Olah, and Jared Kaplan. Language models (mostly) know what they know. *CoRR*, abs/2207.05221, 2022. doi: 10.48550/ARXIV.2207.05221. URL https://doi.org/10.48550/arXiv.2207.05221.

[24] James C. King. Symbolic execution and program testing. *Commun. ACM*, 19(7):385–394, July 1976. ISSN 0001-0782. doi: 10.1145/360248.360252. URL https://doi.org/10.1145/360248.360252.

[25] Tom Kocmi and Christian Federmann. Large language models are state-of-the-art evaluators of translation quality. In Mary Nurminen, Judith Brenner, Maarit Koponen, Sirkku Latomaa, Mikhail Mikhailov, Frederike Schierl, Tharindu Ranasinghe, Eva Vanmassenhove, Sergi Alvarez Vidal, Nora Aranberri, Mara Nunziatini, Carla Parra Escartín, Mikel L. Forcada, Maja Popovic, Carolina Scarton, and Helena Moniz, editors, *Proceedings of the 24th Annual Conference of the European Association for Machine Translation, EAMT 2023, Tampere, Finland, 12-15 June 2023*, pages 193–203. European Association for Machine Translation, 2023. URL https://aclanthology.org/2023.eamt-1.19.

[26] Philipp Koehn and Rebecca Knowles. Six challenges for neural machine translation. In Thang Luong, Alexandra Birch, Graham Neubig, and Andrew M. Finch, editors, *Proceedings of the First Workshop on Neural Machine Translation, NMT@ACL 2017, Vancouver, Canada, August 4, 2017*, pages 28–39. Association for Computational Linguistics, 2017. doi: 10.18653/V1/W17-3204. URL https://doi.org/10.18653/v1/w17-3204.

[27] Lorenz Kuhn, Yarin Gal, and Sebastian Farquhar. Semantic uncertainty: Linguistic invariances for uncertainty estimation in natural language generation. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net, 2023. URL https://openreview.net/forum?id=VD-AYtP0dve.

[28] Salem Lahlou, Moksh Jain, Hadi Nekoei, Victor Ion Butoi, Paul Bertin, Jarrid Rector-Brooks, Maksym Korablyov, and Yoshua Bengio. Deup: Direct epistemic uncertainty prediction. *arXiv preprint arXiv:2102.08501*, 2021.

[29] Yixuan Li, Julian Parsert, and Elizabeth Polgreen. Guiding enumerative program synthesis with large language models. In *International Conference on Computer Aided Verification*, pages 280–301. Springer, 2024.

[30] Stephanie Lin, Jacob Hilton, and Owain Evans. Teaching models to express their uncertainty in words. *Trans. Mach. Learn. Res.*, 2022, 2022. URL https://openreview.net/forum?id=8s8K2UZGTZ.

[31] Fang Liu, Yang Liu, Lin Shi, Houkun Huang, Ruifeng Wang, Zhen Yang, Li Zhang, Zhongqi Li, and Yuchi Ma. Exploring and evaluating hallucinations in llm-powered code generation. *arXiv preprint arXiv:2404.00971*, 2024.

[32] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation. *Advances in Neural Information Processing Systems*, 36, 2024.

[33] Yang Luo, Richard Yu, Fajun Zhang, Ling Liang, and Yongqiang Xiong. Bridging gaps in llm code translation: Reducing errors with call graphs and bridged debuggers. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, pages 2448–2449, 2024.

[34] Andrey Malinin and Mark J. F. Gales. Uncertainty estimation in autoregressive structured prediction. In *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*. OpenReview.net, 2021. URL https://openreview.net/forum?id=jN5y-zb5Q7m.

[35] Sabrina J Mielke, Arthur Szlam, Emily Dinan, and Y-Lan Boureau. Reducing conversational agents' overconfidence through linguistic calibration. *Transactions of the Association for Computational Linguistics*, 10:857–872, 2022.

[36] Kenton Murray and David Chiang. Correcting length bias in neural machine translation. In Ondrej Bojar, Rajen Chatterjee, Christian Federmann, Mark Fishel, Yvette Graham, Barry Haddow, Matthias Huck, Antonio Jimeno-Yepes, Philipp Koehn, Christof Monz, Matteo Negri, Aurélie Névéol, Mariana L. Neves, Matt Post, Lucia Specia, Marco Turchi, and Karin Verspoor, editors, *Proceedings of the Third Conference on Machine Translation: Research Papers, WMT 2018, Belgium, Brussels, October 31 - November 1, 2018*, pages 212–223. Association for Computational Linguistics, 2018. doi: 10.18653/V1/W18-6322. URL https://doi.org/10.18653/v1/w18-6322.

[37] OpenAI. New and improved embedding model. https://openai.com/index/new-and-improved-embedding-model/, December 2022. Accessed: 2025-05-13.

[38] Ian Osband, Zheng Wen, Seyed Mohammad Asghari, Vikranth Dwaracherla, Morteza Ibrahimi, Xiuyuan Lu, and Benjamin Van Roy. Epistemic neural networks. *Advances in Neural Information Processing Systems*, 36:2795–2823, 2023.

[39] Rangeet Pan, Ali Reza Ibrahimzada, Rahul Krishna, Divya Sankar, Lambert Pouguem Wassi, Michele Merler, Boris Sobolev, Raju Pavuluri, Saurabh Sinha, and Reyhaneh Jabbarvand. Lost in translation: A study of bugs introduced by large language models while translating code. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, pages 1–13, 2024.

[40] Dorin Pomian, Abhiram Bellur, Malinda Dilhara, Zarina Kurbatova, Egor Bogomolov, Timofey Bryksin, and Danny Dig. Next-generation refactoring: Combining llm insights and ide capabilities for extract method. In *2024 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pages 275–287. IEEE, 2024.

[41] Shuo Ren, Daya Guo, Shuai Lu, Long Zhou, Shujie Liu, Duyu Tang, Neel Sundaresan, Ming Zhou, Ambrosio Blanco, and Shuai Ma. Codebleu: a method for automatic evaluation of code synthesis. *CoRR*, abs/2009.10297, 2020. URL https://arxiv.org/abs/2009.10297.

[42] Atsushi Shirafuji, Yusuke Oda, Jun Suzuki, Makoto Morishita, and Yutaka Watanobe. Refactoring programs using large language models with few-shot examples. In *2023 30th Asia-Pacific Software Engineering Conference (APSEC)*, pages 151–160. IEEE, 2023.

[43] Ning Tao, Anthony Ventresque, Vivek Nallur, and Takfarinas Saber. Enhancing program synthesis with large language models using many-objective grammar-guided genetic programming. *Algorithms*, 17(7): 287, 2024.

[44] Priyan Vaithilingam, Tianyi Zhang, and Elena L Glassman. Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models. In *Chi conference on human factors in computing systems extended abstracts*, pages 1–7, 2022.

[45] Jiaan Wang, Yunlong Liang, Fandong Meng, Haoxiang Shi, Zhixu Li, Jinan Xu, Jianfeng Qu, and Jie Zhou. Is chatgpt a good NLG evaluator? A preliminary study. *CoRR*, abs/2303.04048, 2023. doi: 10.48550/ARXIV.2303.04048. URL https://doi.org/10.48550/arXiv.2303.04048.

[46] Peiyi Wang, Lei Li, Liang Chen, Zefan Cai, Dawei Zhu, Binghuai Lin, Yunbo Cao, Lingpeng Kong, Qi Liu, Tianyu Liu, and Zhifang Sui. Large language models are not fair evaluators. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2024, Bangkok, Thailand, August 11-16, 2024*, pages 9440–9450. Association for Computational Linguistics, 2024. doi: 10.18653/V1/2024.ACL-LONG.511. URL https://doi.org/10.18653/v1/2024.acl-long.511.

[47] Shangwen Wang, Mingyang Geng, Bo Lin, Zhensu Sun, Ming Wen, Yepang Liu, Li Li, Tegawendé F. Bissyandé, and Xiaoguang Mao. Natural language to code: How far are we? In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ESEC/FSE 2023, page 375–387, New York, NY, USA, 2023. Association for Computing Machinery. ISBN 9798400703270. doi: 10.1145/3611643.3616323. URL https://doi.org/10.1145/3611643.3616323.

[48] Tianlu Wang, Ping Yu, Xiaoqing Ellen Tan, Sean O'Brien, Ramakanth Pasunuru, Jane Dwivedi-Yu, Olga Golovneva, Luke Zettlemoyer, Maryam Fazel-Zarandi, and Asli Celikyilmaz. Shepherd: A critic for language model generation. *CoRR*, abs/2308.04592, 2023. doi: 10.48550/ARXIV.2308.04592. URL `https://doi.org/10.48550/arXiv.2308.04592`.

[49] Yidong Wang, Zhuohao Yu, Zhengran Zeng, Linyi Yang, Cunxiang Wang, Hao Chen, Chaoya Jiang, Rui Xie, Jindong Wang, Xing Xie, Wei Ye, Shikun Zhang, and Yue Zhang. Pandalm: An automatic evaluation benchmark for LLM instruction tuning optimization. *CoRR*, abs/2306.05087, 2023. doi: 10.48550/ARXIV.2306.05087. URL `https://doi.org/10.48550/arXiv.2306.05087`.

[50] Tingyu Xia, Bowen Yu, Yuan Wu, Yi Chang, and Chang Zhou. Language models can evaluate themselves via probability discrepancy. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Findings of the Association for Computational Linguistics, ACL 2024, Bangkok, Thailand and virtual meeting, August 11-16, 2024*, pages 4889–4901. Association for Computational Linguistics, 2024. doi: 10.18653/V1/2024.FINDINGS-ACL.291. URL `https://doi.org/10.18653/v1/2024.findings-acl.291`.

[51] Gal Yona, Roee Aharoni, and Mor Geva. Narrowing the knowledge evaluation gap: Open-domain question answering with multi-granularity answers. *arXiv preprint arXiv:2401.04695*, 2024.

[52] Hanliang Zhang, Cristina David, Wang Meng, Brandon Paulsen, and Daniel Kroening. Scalable, validated code translation of entire projects using large language models. *Programming Language Design and Implementation (PLDI)*, 2025.

[53] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. Autocoderover: Autonomous program improvement. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, ISSTA 2024, page 1592–1604, New York, NY, USA, 2024. Association for Computing Machinery. ISBN 9798400706127. doi: 10.1145/3650212.3680384. URL `https://doi.org/10.1145/3650212.3680384`.

A Semantic Uncertainty via Symbolic Clustering

This section presents our adaptation of the semantic entropy-based approach by Kuhn et al. [27] for code generation. We follow the main steps from the original work while diverging in two key aspects: the way we estimate the distribution of generated LLM responses and the clustering methodology.

A.0.1 Generation

The first step involves sampling M code snippets (using the same hyperparameters as Kuhn et al. [27]), $\{s^{(1)}, \ldots, s^{(M)}\}$, from the LLM's output distribution $p(s \mid x)$ for a given prompt x. The probabilities of the collected samples are processed using a softmax-style normalization function, ensuring that the resulting values can be interpreted as a valid probability distribution.

Given that this process can lead to numerical underflows, to mitigate this, we approximate the probability distribution of the LLM responses using either length-normalization or a uniform distribution. Following this approximation, let $\tilde{p}(s \mid x)$ denote the probability of a snippet s according to the adjusted distribution.

A.0.2 Clustering via Symbolic Execution

The second step works by grouping the aforementioned snippets into clusters based on semantic equivalence. This process, as shown in Algorithm 1 from Section 4, is based on symbolic execution.

A.0.3 Entropy Estimation

The final step computes uncertainty as the semantic entropy over clusters, reflecting the diversity of functional behaviours.

First, the probability associated with a cluster c is calculated as:

$$\tilde{p}(c \mid x) = \sum_{s \in c} \tilde{p}(s \mid x),$$

where $s \in c$ indicates that the snippet s belongs to the cluster c. Then, the entropy $H(C \mid x)$ over the set of clusters C is defined as:

$$H(C \mid x) = -\sum_{c \in C} \log \tilde{p}(c \mid x),$$

where C denotes all semantic clusters obtained from Algorithm 1 in Section 4. A higher entropy indicates greater semantic diversity and hence higher uncertainty in the functional behaviour captured by the clusters. Conversely, a lower entropy suggests that the model's outputs are concentrated around a few semantically equivalent behaviours, reflecting higher confidence.

B Mutual Information Estimation via Symbolic Clustering

This section presents an adaptation of the mutual information-based approach for quantifying epistemic uncertainty by Abbasi et al. [3] to the domain of code generation. We follow the steps from the original work: iterative prompting for generating LLM responses, clustering, and mutual information estimation. However, similar to Section A, we diverge with respect to the methodology for clustering responses and the way we estimate the distribution of generated LLM responses.

B.0.1 Iterative Prompting for Code Generation

Iterative prompting is used for generating multiple responses from the LLM and consequently in constructing a pseudo joint distribution of outputs.

More precisely, the LLM is sampled to produce n responses while also getting their respective probabilities, $\mu(X_j)$ for $j = 1, 2, \ldots, n$. These responses are first used to construct iterative prompts by appending the response to the original prompt and asking the LLM to produce more responses. This step then makes use of softmax-style normalization to obtain values that can then be treated as probabilities which are used in the subsequent steps.

As before since this process can lead to numerical underflows, to mitigate this, we use length-normalization. As opposed to the approach in Section A, here we did not use the uniform distribution approximation, as the actual LLM-reported probabilities are needed to distinguish between aleatoric and epistemic uncertainties.

Following length normalization, we compute conditional probabilities, $\mu(X_m|X_n)$ for $m, n = 1, 2, \ldots, n$, by looking at the response probabilities received from the LLM when subjected to the aforementioned iterative prompts.

B.0.2 Clustering via Symbolic Execution

To handle functional diversity, the generated program snippets are clustered based on their semantic equivalence using Algorithm 1.

B.0.3 Mutual Information Estimation

Once clustering is complete, mutual information is computed over the resulting clusters to quantify epistemic uncertainty.

The aggregated probabilities are defined as:

$$\mu_1'(X_i) = \sum_{j \in D(i)} \mu(X_j), \quad \mu_2'(X_t \mid X_i) = \sum_{j \in D(t)} \mu(X_j \mid X_i),$$

where X_i and X_t are clusters, $\mu(X_j)$ represents the probability of the output X_j, and $\mu(X_j \mid X_i)$ is the conditional probability of X_j given X_i. The set $D(i)$ contains all outputs assigned to the cluster X_i.

The normalised empirical distributions are:

$$\hat{\mu}_1(X_i) = \frac{\mu_1'(X_i)}{Z}, \quad \text{where} \quad Z = \sum_{j \in S} \mu_1'(X_j),$$

$$\hat{\mu}_2(X_t \mid X_i) = \frac{\mu_2'(X_t \mid X_i)}{Z_i}, \quad \text{where} \quad Z_i = \sum_{j \in S} \mu_2'(X_j \mid X_i).$$

Here, $\hat{\mu}_1(X_i)$ is the normalised marginal distribution for cluster X_i, and $\hat{\mu}_2(X_t \mid X_i)$ is the normalised conditional distribution for X_t given X_i. The terms Z and Z_i are normalization constants to ensure that the distributions sum to 1.

The joint and pseudo-joint distributions are defined as:

$$\hat{\mu}(X_i, X_t) = \hat{\mu}_1(X_i)\, \hat{\mu}_2(X_t \mid X_i),$$
$$\hat{\mu}^{\otimes}(X_i, X_t) = \hat{\mu}_1(X_i) \sum_{j \in S} \hat{\mu}_1(X_j)\, \hat{\mu}_2(X_t \mid X_j).$$

The joint distribution $\hat{\mu}(X_i, X_t)$ combines the marginal and conditional distributions, while the pseudo-joint distribution $\hat{\mu}^{\otimes}(X_i, X_t)$ assumes independence between clusters.

Finally, the mutual information is computed as:

$$\hat{I}(\gamma_1, \gamma_2) = \sum_{i,t \in S} \hat{\mu}(X_i, X_t) \ln \left(\frac{\hat{\mu}(X_i, X_t) + \gamma_1}{\hat{\mu}^{\otimes}(X_i, X_t) + \gamma_2} \right).$$

Here, γ_1 and γ_2 are small stabilization parameters to prevent division by zero, and S is the set of clusters.

This mutual information score serves as a proxy for epistemic uncertainty. High $\hat{I}$ values signal significant uncertainty.

C Additional Evaluation

C.1 Experimental Results for `claude-3-7-sonnet`

Table 4: Correlation Results — Pearson coefficient (p-value) for `claude-3-7-sonnet`. Statistically significant results are highlighted in **bold**.

Technique	`claude-3-7-sonnet`		
	Easy	**Medium**	**Hard**
SE-SYMBUNIFORM	**−0.35(0.0002)**	**−0.34(0.0002)**	**−0.32(0.0003)**
CC-SYMB	**−0.35(0.0002)**	**−0.34(0.0002)**	**−0.32(0.0003)**

C.2 Details on the Abstention Policy Evaluation

Given that `gpt-3.5-turbo-instruct`'s responses exhibit a low unit test passing rate, leading to an imbalanced dataset where incorrect responses significantly outnumber correct ones. This imbalance risks trivializing the classification task, as an "all-incorrect" model would dominate. To address this, we followed standard practice and applied random downsampling, removing a portion of incorrect responses to prevent the model from defaulting to a trivial solution. We split the dataset 50/50 into training and validation. To mitigate overfitting and reduce bias, we employed 2-fold cross-validation, alternating between training and validation for each fold.